Adaptive Timing Control for Throughput–Latency Trade-off in nFAPI Split

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1†
*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan
†Affiliation 2
Email: crg@mail.ntust.edu.tw

***Abstract*—Decoupling the Medium Access Control (MAC) and Physical (PHY) layers in the nFAPI (Option 6) functional split introduces severe timing synchronization challenges over packet-switched networks. To prevent message rejection at the physical layer (PNF), the virtualized MAC (VNF) must schedule and transmit control messages ahead of time. This paper proposes an adaptive timing control framework where the VNF dynamically adjusts this scheduling lead time (slots ahead) using closed-loop feedback based on real-time Timing Info reported by the PNF. By continuously updating the slots ahead, the system automatically adapts to diverse deployment environments with dynamic network congestion, including varying offered loads, latencies, and jitters. Experimental validation on an OpenAirInterface (OAI) testbed integrated with a commercial Radio Unit (O-RU) demonstrates that the proposed mechanism effectively eliminates packet errors, stabilizes round-trip times (RTT), and maintains robust throughput under severe network congestion.**

***Index Terms*—nFAPI, O-RAN, Functional Split, Adaptive Control.**

I. Introduction

A. Background and Motivation

To address the diverse demands of 5G networks, the 3rd Generation Partnership Project (3GPP) and the O-RAN Alliance have standardized the disaggregation of traditional base stations into Open Central Units (O-CUs), Open Distributed Units (O-DUs), and Open Radio Units (O-RUs) [1]. Among the proposed architectures, three functional split configurations have emerged as mainstream: Option 2 (O-CU/O-DU), Option 7.2 (O-DU/O-RU), and Option 6 (O-DU High/Low) [2].

These splits present distinct synchronization requirements and management strategies. Option 2 relies on sequence numbers and deep buffering over general packet-switched networks, making its timing constraints relatively relaxed. Option 7.2 [3], operating within the physical layer, demands strict phase/symbol-level synchronization enforced by hardware Precision Time Protocol (PTP) over Time-Sensitive Networking (TSN). Occupying a unique intermediate position is Option 6 (the MAC-PHY split). Unlike the FAPI standard which assumes same-machine shared-memory deployment driven by `slot.indication`, Option 6 is standardized as nFAPI by the Small Cell Forum (SCF) [4] to support multi-machine socket deployment. Option 6 requires frame/slot-level synchronization and focuses on delay management, offering the flexibility to operate over general packet-switched networks without the very high costs of TSN.

The rationale for deploying the MAC layer (O-DU High) in a cloud environment via Option 6 is primarily driven by resource pooling, as Layer 2 processing complexity scales dynamically with user equipment (UE) density [5]. However, migrating the MAC layer to a Virtualized Network Function (VNF) introduces internal processing latency that compresses the available margin for fronthaul transmission delay. Unlike Option 7.2, which rigidly drops out-of-window packets, Option 6 must adaptively manage delays to ensure slot-level synchronization over non-deterministic Ethernet transport. Existing open-source implementations typically employ static slots ahead configurations, which fail to accommodate dynamic network jitter and server processing variations, leading to frequent scheduling failures (e.g., missing slot packet).

This highlights a fundamental gap in current disaggregated RAN designs: the lack of adaptive timing control mechanisms for intermediate-layer splits operating under non-ideal transport environments.

B. Related Works

Recent advancements in Open RAN (O-RAN) architectures have driven significant research into optimizing split-architecture performance and managing network latency. For instance, the X5G testbed proposed by Villa *et al.* [6] demonstrates a state-of-the-art approach by deploying a hardware-accelerated private 5G O-RAN network. By offloading Physical (PHY) layer processing to dedicated NVIDIA GPUs and Mellanox SmartNICs, X5G successfully achieves commercial-grade peak throughputs. However, this high performance relies heavily on expensive, specialized hardware and demands strict, ideal network conditions equipped with hardware-level Precision Time Protocol (PTP) synchronization. Such stringent requirements significantly limit its applicability and deployment flexibility in general packet-switched networks, where unpredictable latency and jitter are prevalent. To guide the transition of these complex virtualized architectures from simulators to emulated and real-world O-RAN testbeds, deployment guidelines and interoperability challenges using open-source software such as OpenAirInterface have been documented [7].

Beyond hardware acceleration, software-driven delay management strategies have also been explored from various perspectives. At the intra-node computational level, Lee *et al.* [8] proposed an adaptive Layer 1 transmission strategy that opportunistically advances transmissions when software processing finishes early. To complement this, Aquifer [9] introduced transparent microsecond-scale OS scheduling to mitigate processing delay variations in vRAN workloads. From a network orchestration perspective, Adamuz-Hinojosa *et al.* [10] introduced ORANUS, employing Stochastic Network Calculus (SNC) to provide mathematical latency bounds for ultra-reliable low-latency communications (uRLLC). Moreover, Lv *et al.* [11] emphasized User Equipment (UE) QoS support by integrating MAC resource allocation with high-level network orchestration. Additionally, the efficiency of mobile scheduler resource allocation is heavily impacted by HARQ retransmission delays, where dynamic pre-allocation schemes have been proposed to minimize block loss rate and stabilize link-layer throughput [12]. To improve energy efficiency, frameworks like AegisRAN [13] dynamically scale vRAN CPU resources, though deactivating idle cores introduces transient processing delay spikes that further complicate timing synchronization. While these frameworks [8]–[13] provide robust solutions for processing variance, capacity bounds, energy conservation, and QoS scheduling, they do not directly address the critical synchronization failures caused by unpredictable fronthaul network jitter in intermediate-layer splits.

Fronthaul transmission delays and capacity limits have also been modeled using queuing theory. Diez *et al.* [14] developed an end-to-end queuing model to characterize fronthaul latency across flexible splits, demonstrating how background traffic dramatically inflates delay. This was further extended by Khan *et al.* [15] to evaluate time-window boundaries and latency budget constraints in spine-leaf fronthaul topologies.

To address the limitations of high hardware costs and strict synchronization requirements, this paper proposes a cost-effective, software-based solution tailored for non-ideal fronthaul environments. Specifically, we focus on the fact that the Small Cell Forum (SCF) 225 nFAPI specification [4] defines the standard `Timing Info` message and interface-level delay management fields, yet existing open-source implementations only support static parameter initialization. Our work introduces an adaptive slots ahead timing control mechanism based on Exponentially Weighted Moving Average (EWMA) delay management to complete this standardized dynamic feedback loop. This approach dynamically tracks and filters network jitter using general-purpose Commercial Off-the-Shelf (COTS) servers. Experimental results demonstrate that our proposed method ensures stable Hybrid Automatic Repeat Request (HARQ) Round-Trip Times (RTT) and stable system throughput without the need for high-end specialized equipment or strict hardware synchronization, thereby offering a more flexible and economically viable alternative for O-RAN deployments.

Specifically, compared to Split 7.2 implementations like X5G [6] that require hardware-level PTP synchronization and GPU/SmartNIC offloads, or FAPI-based Split 6 designs [8] restricted to intra-node shared memory and process-level adjustments, our proposed framework operates at the slot/TTI level on standard COTS servers, actively managing cross-node network jitter and delay over standard packet-switched Ethernet without hardware acceleration dependencies.

C. Research Scope and Contributions

In this paper, we focus on the nFAPI-based Option 6 split as a representative case of timing-sensitive yet transport-flexible RAN disaggregation. While Option 6 offers significant advantages in terms of reduced fronthaul bandwidth and deployment flexibility, its performance is highly sensitive to non-deterministic latency introduced by transport networks and server processing.

We identify that existing open-source implementations, including OpenAirInterface (OAI), do not implement a generic and spec-compliant nFAPI interface according to the SCF standards. Instead, they use an incorrect approach where the raw `slot.indication` is forwarded directly over the network, and they rely on a statically configured slots ahead parameter. This static approach leads to high latency and makes the system highly sensitive to packet-switched network jitter. To address this issue, we re-develop the nFAPI interface based on the OAI project to support the standard Node sync and Delay Management mechanisms. By providing generic inputs and outputs rather than hardcoded slot forwarding, our framework enables dynamic timing control and flexible deployment. We also plan to contribute this re-developed nFAPI implementation back to the upstream OAI repository.

To address this issue, we propose an adaptive timing control framework that dynamically adjusts scheduling lead time based on real-time latency estimation. The proposed approach consists of two main components: Node sync, which aligns frame and slot boundaries between the VNF and PNF, and Delay Management, which applies an Exponentially Weighted Moving Average (EWMA) to smooth the timing information and dynamically adjust the slots ahead to ensure that scheduling messages arrive within the valid timing window.

The main contributions of this paper are summarized as follows:

- **Systematic Analysis of Timing Uncertainty in Split 6:** We characterize the impact of non-deterministic latency, including network jitter and processing delay, on synchronization stability in disaggregated RAN systems.
- **Adaptive Slots Ahead mechanism:** We propose a dynamic timing adjustment mechanism that continuously aligns scheduling decisions with varying transport delays, effectively mitigating timing violations.
- **Experimental Validation on OAI Platform:** We implement the proposed framework in a real-world testbed and demonstrate that it eliminates missing slot packet errors and significantly improves throughput stability under non-ideal fronthaul conditions.
- **Open-Source nFAPI Platform Contribution to Upstream OAI:** We submitted our adaptive timing control and nFAPI modifications as a merge request (MR) to the official OpenAirInterface (OAI) repository. This contribution was merged into the official codebase. It provides a standard open-source nFAPI platform that

follows the Small Cell Forum (SCF) specification for future development and testing.

II. System Model

Before diving into the system details, this study introduces the adopted system framework. To integrate the Network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the proposed system adopts a stable two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to a commercial Pegatron O-RU. Figure 1 illustrates this proposed system architecture and its delay management mechanism.

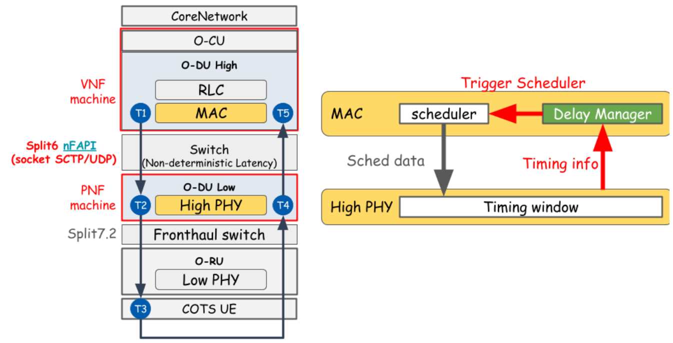


Fig. 1. System architecture and delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user equipment:

- **Core Network (CN):** The central element responsible for routing, session management, and authenticating user data.
- **O-RAN Central Unit (O-CU):** A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.
- **O-RAN Distributed Unit (O-DU):** To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.
- **Switch:** A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.
- **O-RAN Radio Unit (O-RU):** A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.
- **Commercial Off-The-Shelf User Equipment (COTS UE):** Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC, Scheduler, and Timing Core

Within the O-DU High virtualized network function, the timing control is organized as a unified timing control system. The core processes handling this timing core include the Scheduler, Node sync, and Delay Management:

- **Scheduler:** Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.
- **Node sync:** Responsible for software-level clock synchronization. This procedure periodically exchanges synchronization timestamps ($t_1 \sim t_4$) with the PNF via standard *DL/UL_node_sync* messages. It calculates the instantaneous clock offset and applies slot-level (S_{adj}) and microsecond-level (μs_{adj}) adjustments to align the VNF's local clock to the PNF baseline, eliminating long-term hardware clock drift.
- **Delay Management:** Operates on top of the stabilized clock phase established by Node sync. This mechanism utilizes the arrival offset (Δt_{arrive}) feedback from the PNF to dynamically estimate transport network jitter. It then actively adjusts the slots ahead (S_{ahead}) trigger offset of the scheduler. This control compensates for the non-deterministic packet transport delays and VNF processing delays without inducing synchronization loss.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- **Timing Window:** A predefined, strict time duration. Let T_{window} denote the duration of this PNF timing window, and T_{slot} denote the subframe slot duration. When the scheduling message (specifically the `DL_TTI.request`) sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the slot data.
- **Timing Info (Δt_{arrive}):** The PNF continuously monitors the exact arrival offset of control messages relative to the transmission deadline. This offset, denoted as Δt_{arrive}, represents the remaining time margin within the timing window. The PNF periodically sends this information back to the MAC layer for Delay Management. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal environments, this study defines the following Key Validation Metrics:

- **MAC Layer HARQ RTT ($T_5 - T_1$):** Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically at the MAC layer. This metric accurately reflects the feedback efficiency and operational health of the link layer.
- **VNF-PNF DL Latency ($T_2 - T_1$):** Measures the one-way downlink latency accumulated from the virtualized machine (VNF) to the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

Note that RTT denotes the Round-Trip Time. To validate the split-architecture stability under non-ideal fronthaul conditions, network latency simulation is conducted at the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments via Linux Traffic Control (TC).

To validate the proposed performance optimization within a realistic network environment, the system design leverages a commercial-grade deployment framework:

- **O-RAN Split 7.2x Interoperability:** While the Small Cell Forum (SCF) has standardized the nFAPI for the MAC-PHY split (Option 6) [16], commercial-grade Open Radio Units (O-RUs) natively support the O-RAN Split Option 7.2x. To achieve seamless integration with open-source stacks like OpenAirInterface [17], our system bridges these interfaces.
- **Hybrid System Architecture for E2E Validation:** To validate the nFAPI performance over a fully commercial ecosystem, this paper adopts a hybrid architecture (Split Option 6 + Split Option 7.2). This setup establishes a translation/interworking layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with a commercial Pegatron O-RU, facilitating end-to-end verification using Commercial Off-The-Shelf (COTS) UEs.

III. Proposed Method

This section details the proposed dynamic timing adjustment framework. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose two core timing control algorithms: Node sync for software-level clock synchronization and Delay Management for slot-level adaptive delay management. The timing control of this system is organized as a closed-loop system, ensuring that the VNF-side scheduling can dynamically track and compensate for both clock drift and slot-level transport latency variation. The two algorithms cooperate as follows:

1) **Node sync:** Maintains phase and slot boundary alignment between the VNF and PNF. It periodically exchanges timestamps to calculate the absolute phase offset, correcting for hardware clock drift.
2) **Delay Management:** Tracks and estimates packet transport delays and jitter at the slot level, dynamically adjusting the slots ahead (S_{ahead}) to guarantee that control messages fall within the PNF's strict timing window.

A. Node sync

To perform software-level clock synchronization, the system adopts a timestamp exchange mechanism. The VNF periodically transmits a downlink node synchronization message with a local timestamp t_1. Upon receiving the message, the PNF records the local arrival timestamp t_2 and transmits an uplink node synchronization message back to the VNF, including the transmit timestamp t_3 and the echoed t_2. When the VNF receives the message, it records its local arrival timestamp t_4. Because the nFAPI timestamps are constrained by a 1024 subframe number (SFN) loop, they wrap around every 10.24 seconds (10240000 µs). The algorithm calculates the differences $d_1 = t_2 - t_1$ and $d_2 = t_4 - t_3$, applying a wrap-around correction to align them within the interval $[-5120000, 5120000]$ µs. The VNF then calculates the instantaneous clock offset:

$$offset = \frac{d_1 - d_2}{2} \tag{1}$$

To prevent timing adjustments from causing control loop oscillations, the synchronization control loop implements a state-based locking mechanism. The system has two synchronization states: locked ($sync\_locked = 1$) and unlocked ($sync\_locked = 0$). In the locked state, the VNF monitors the drift of the local clock. If the absolute offset exceeds the slot duration T_{slot}:

$$|offset| > T_{\text{slot}} \tag{2}$$

the VNF unlocks the synchronization ($sync\_locked = 0$) to trigger re-synchronization. In the unlocked state, the VNF checks whether the offset has converged within the slot duration T_{slot}. If $|offset| \leq T_{\text{slot}}$, the system locks the synchronization state ($sync\_locked = 1$). Otherwise, the VNF decomposes the offset into a slot-level adjustment S_{adj} and a microsecond-level adjustment μs_{adj} based on the slot duration T_{slot}:

$$S_{\text{adj}} = offset / T_{\text{slot}} \tag{3}$$

$$\mu s_{\text{adj}} = offset \pmod{T_{\text{slot}}} \tag{4}$$

These adjustments are accumulated into the global adjustment variables:

$$slot\_adjustment \leftarrow slot\_adjustment + S_{\text{adj}} \tag{5}$$

$$us\_adjustment \leftarrow us\_adjustment - \mu s_{\text{adj}} \tag{6}$$

The negative sign in the microsecond adjustment reduces the local thread sleep time when the VNF clock is behind, speeding up the slot transition rate to catch up. The synchronization process is summarized in Algorithm 1.

Algorithm 1 Node sync

Input: t_1, t_2, t_3 (Timestamps from PNF *UL_node_sync*), t_4 (VNF local receive timestamp)

Output: $slot\_adjustment$ (Slot adjustment accumulator), $us\_adjustment$ (Microsecond adjustment accumulator)

Notation:

offset (Instantaneous timing offset), *sync_locked* (Synchronization lock state)

T_{slot} (Slot duration in µs), T_{wrap} (Wrap-around period, 10240000 µs)

WrapCorrection(x, T) (Corrects x within $[-T/2, T/2]$)

Phase I: Timing Offset Calculation

$d_1 \leftarrow \text{WrapCorrection}(t_2 - t_1, T_{\text{wrap}})$

$d_2 \leftarrow \text{WrapCorrection}(t_4 - t_3, T_{\text{wrap}})$

$\mathit{offset} \leftarrow (d_1 - d_2)/2$

Phase II: Hysteresis Lock and Clock Adjustment

if $\mathit{sync\_locked} = 1 \wedge |\mathit{offset}| > T_{\text{slot}}$ **then**

  $\mathit{sync\_locked} \leftarrow 0$ ▷ Unlock due to excessive clock drift

end if

if $\mathit{sync\_locked} = 0$ **then**

  if $|\mathit{offset}| \leq T_{\text{slot}}$ **then**

    $\mathit{sync\_locked} \leftarrow 1$ ▷ Lock synchronization

  else

    $S_{\text{adj}} \leftarrow \mathit{offset}/T_{\text{slot}}$ ▷ Integer division

    $\mu s_{\text{adj}} \leftarrow \mathit{offset} \pmod{T_{\text{slot}}}$

    $slot\_adjustment \leftarrow slot\_adjustment + S_{\text{adj}}$

    $us\_adjustment \leftarrow us\_adjustment - \mu s_{\text{adj}}$

  end if

end if

return $slot\_adjustment, us\_adjustment$

B. Delay Management

To counteract changing network latency and maintain scheduling stability, a dynamic delay management strategy is implemented. This strategy operates as a Fast-Up and Slow-Down Feedback Controller based on a Worst-Case Delay Model, periodically evaluating the observed arrival offset (Δt_{arrive}) reported by the PNF.

1) Delay and Uncertainty Estimation: To smooth high-frequency network noise while remaining sensitive to timing drift, Delay Management maintains a running mean of the arrival offset (μ_{arrive}) and the arrival jitter (dev_{arrive}). Let $\Delta t_{\text{arrive},i}$ be the observed arrival offset in scheduling slot i. The EWMA updates for mean arrival offset and jitter are defined as:

$$\mu_{\text{arrive},i} = \mu_{\text{arrive},i-1} + \alpha \cdot (\Delta t_{\text{arrive},i} - \mu_{\text{arrive},i-1}) \tag{7}$$

$$dev_{\text{arrive},i} = dev_{\text{arrive},i-1} + \beta \cdot (|\Delta t_{\text{arrive},i} - \mu_{\text{arrive},i}| - dev_{\text{arrive},i-1}) \tag{8}$$

where the smoothing factors are chosen as $\alpha = 1/8$ to track the long-term trend, and $\beta = 1/4$ to capture fast jitter variations. These parameters are designed as negative powers of two ($\alpha = 2^{-3}$, $\beta = 2^{-2}$) to allow the controller to perform fast bitwise shift operations in real time.

To guide timing adjustments, the controller combines the mean arrival offset and the estimated jitter to estimate the worst-case timing delay (D_{worst}). The worst-case timing delay for slot i is calculated as the sum of the worst-case boundary of the arrival offset and the arrival jitter:

$$D_{\text{worst},i} = \max(\Delta t_{\text{arrive},i}, \mu_{\text{arrive},i}) + dev_{\text{arrive},i} \tag{9}$$

Based on this worst-case delay estimation, the controller recursively tracks the accumulated timing delay (D_{accum}):

$$D_{\text{accum},i} = \max(0, D_{\text{accum},i-1} + D_{\text{worst},i}) \tag{10}$$

The accumulated timing delay D_{accum} aggregates timing violations and worst-case delays over time. The system is considered delay-free if D_{accum} is below the arrival jitter level ($D_{\text{accum}} \leq dev_{\text{arrive}}$).

2) Fast-Up and Slow-Down Feedback Control: The update of the slots ahead trigger timing S_{next} is governed by a fast-up and slow-down control law. Under this law, the scheduler reacts immediately to timing violations or positive timing delay, while slowly decreasing the lead time only when the system has remained stable for a sustained duration:

- **Rapid Increase (Preserve Throughput):** If a timing violation is detected ($\Delta t_{\text{arrive}} > 0$) or the worst-case timing delay is positive ($D_{\text{worst}} > 0$), the controller immediately increments the slots ahead parameter to prevent scheduling timeouts:

$$S_{\text{next}} = \min\left(S_{\text{curr}} + 1, \left\lfloor \frac{T_{\text{window}}}{T_{\text{slot}}} \right\rfloor\right) \tag{11}$$

  and resets the stable observation counter $C_{\text{stable}} = 0$.
- **Gradual Decrease (Optimize Latency):** If the system's accumulated timing delay is below the jitter level ($D_{\text{accum}} \leq dev_{\text{arrive}}$) and the current worst-case timing delay is safe ($D_{\text{worst}} \leq -dev_{\text{arrive}}$), the controller decrements the slots ahead timing once the stability condition is satisfied:

$$S_{\text{next}} = \max(S_{\text{curr}} - 1, 1) \tag{12}$$

  and resets $C_{\text{stable}} = 0$. To prevent premature timing reductions as the scheduling buffer grows, the required stable count threshold increases quadratically with S_{curr}.
- **Maintain Baseline:** Otherwise, the controller holds the current timing, $S_{\text{next}} = S_{\text{curr}}$. The stable counter C_{stable} is incremented if the current sample is safe ($D_{\text{worst}} \leq 0$).

The core adjustment strategy is condensed in Algorithm 2.

C. Coordination between Node sync and Delay Management

To ensure stable operation under real-world conditions, Node sync and Delay Management must coordinate without inducing control loop oscillations. The microsecond-level Node sync focuses strictly on correcting clock phase offsets to align the VNF's scheduling thread with the PNF's slot transitions. Because it directly shifts the VNF's time base, any clock adjustment ($S_{adj}, \mu s_{adj}$) temporarily shifts the arrival timestamp reference. Delay Management is designed to absorb

Algorithm 2 Delay Management

Input: Δt_{arrive} (Observed arrival offset), S_{curr} (Current trigger timing), T_{window} (Timing window), T_{slot} (Slot duration)

Output: S_{next} (Trigger timing for the next MAC schedule)

Notation:
μ_{arrive} (smoothed mean arrival offset), dev_{arrive} (smoothed arrival jitter)
D_{worst} (Worst-case timing delay), D_{accum} (Accumulated timing delay)
C_{stable} (Stable observation counter)
α, β (Smoothing factors for latency and jitter)

Phase I: Latency and Jitter Estimation

$\mu_{\text{arrive}} \leftarrow \mu_{\text{arrive}} + \alpha \cdot (\Delta t_{\text{arrive}} - \mu_{\text{arrive}})$ ▷ Update mean arrival offset
$dev_{\text{arrive}} \leftarrow dev_{\text{arrive}} + \beta \cdot (|\Delta t_{\text{arrive}} - \mu_{\text{arrive}}| - dev_{\text{arrive}})$ ▷ Update arrival jitter
$D_{\text{worst}} \leftarrow \max(\Delta t_{\text{arrive}}, \mu_{\text{arrive}}) + dev_{\text{arrive}}$ ▷ Evaluate worst-case timing delay
$D_{\text{accum}} \leftarrow \max(0, D_{\text{accum}} + D_{\text{worst}})$ ▷ Update accumulated delay

Phase II: Fast-Up and Slow-Down Timing Adjustment

if $\Delta t_{\text{arrive}} > 0 \lor D_{\text{worst}} > 0$ **then**
    $S_{\text{next}} \leftarrow \min(S_{\text{curr}} + 1, \lfloor T_{\text{window}} / T_{\text{slot}} \rfloor)$ ▷ Rapid Increase
    $C_{\text{stable}} \leftarrow 0$
else if $D_{\text{accum}} \leq dev_{\text{arrive}} \land D_{\text{worst}} \leq -dev_{\text{arrive}}$ **then**
    if stability condition is satisfied **then**
        $S_{\text{next}} \leftarrow \max(S_{\text{curr}} - 1, 1)$ ▷ Gradual Decrease
        $C_{\text{stable}} \leftarrow 0$
    else
        $S_{\text{next}} \leftarrow S_{\text{curr}}$
        $C_{\text{stable}} \leftarrow C_{\text{stable}} + 1$
    end if
else
    $S_{\text{next}} \leftarrow S_{\text{curr}}$
    $C_{\text{stable}} \leftarrow 0$
end if
return S_{next}

this transient shifting. By relying on the history-weighted accumulated timing delay (D_{accum}), Delay Management avoids reacting to the temporary timing offsets caused by clock adjustments, allowing Node sync to converge. Once Node sync locks synchronization ($sync\_locked = 1$), the time base stabilizes, allowing Delay Management to achieve precise slot-level optimizations (S_{ahead}).

IV. Experimental Results

Experiments were conducted to verify the proposed delay management method in an OpenAirInterface (OAI) nFAPI End-to-End environment. Each reported throughput value represents the average of 180 data points (3 independent trials $\times$ 60 samples per trial). We structure the evaluation into three parts: validating the EWMA mechanism (Section IV-C), benchmarking peak throughput and MAC Round-Trip Time (RTT) against static baselines (Section IV-D), and evaluating stability under network congestion (Section IV-E).

A. Experimental Scenario and rApp Framework

The testbed environment connects a series of high-performance servers, radio units, and a commercial smartphone. The detailed software and hardware configurations for the end-to-end network deployment are outlined below.

TABLE I
Testbed Node Configuration

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (`2026.w13`)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE II
Wireless System Parameters

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

To ensure automated measurement and analysis with consistent data quality, an automated evaluation rApp was developed to operate on the Non-Real-Time RAN Intelligent Controller (Non-RT RIC) within the O-RAN Service Management and Orchestration (SMO) framework [18]. As shown in Fig. 2, this rApp automates the start-up of the VNF and PNF. Once the gNB is established, it automatically connects to the control PC via SSH to toggle the UE's airplane mode using Android Debug Bridge (ADB), and subsequently uses SSH to launch the iPerf server on the Core Network (CN) server. It then executes a series of pre-scheduled iPerf tests automatically. Finally, the rApp collects all log files via O1 SFTP and utilizes Python scripts to plot the following data analysis charts. Crucially, it manages the network latency simulation at $T_5 - T_4$ and $T_2 - T_1$ via Linux Traffic Control (TC) to systematically validate the split-architecture stability.

B. Rigorous System-Level Validation and Generalization

To prove the stability, generalization capability, and compatibility of the proposed timing control and nFAPI modifications, our software implementation was submitted and successfully merged into the official upstream OpenAirInterface (OAI) repository. Beyond our localized end-to-end evaluation using the rApp framework, the proposed solution underwent rigorous testing in the official OAI Continuous Integration and Continuous Deployment (CI/CD) pipeline. Specifically, the code successfully passed all image building and cross-compilation

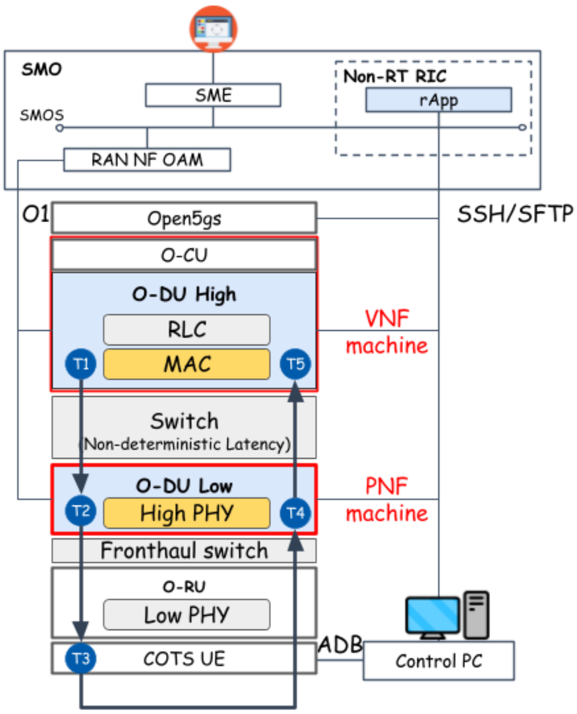
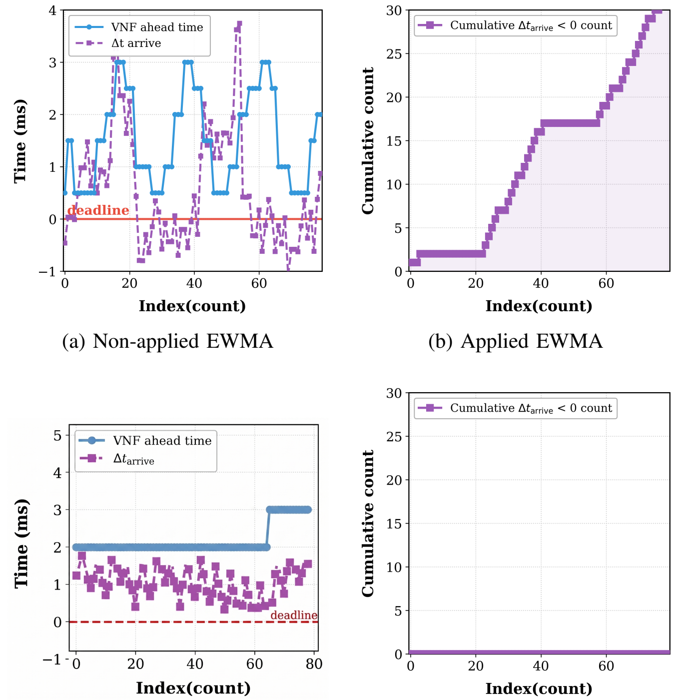


Fig. 2. Architecture of the rApp automated evaluation framework.



(a) Non-applied EWMA (b) Applied EWMA

(c) Trend Detection (d) Proactive Adjustment

Fig. 3. Validation of EWMA processing on PNF Timing Info and proactive adjustment.

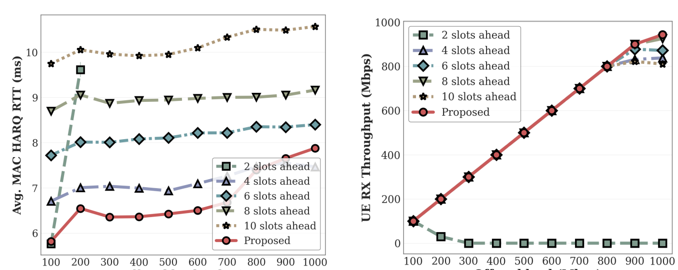
tests across standard x86 and ARM64-based architectures, including enterprise-grade Red Hat Enterprise Linux (RHEL) clusters and NVIDIA Jetson edge platforms. Furthermore, our implementation successfully completed key system-level integration and physical layer regression tests. These tests include 5G Standalone (SA) end-to-end integration, real radio frequency (RF) transmissions using USRP B200 and AW2S radio units, standard F1/N2 interface control-plane handover procedures, and 3GPP-compliant physical layer channel simulations. These regression and compilation results confirm that our proposed software-defined timing loops are highly robust, backward-compatible, and ready for diverse commercial cloud hardware deployments without introducing any hardware dependency.

C. Scenario I: Validation of EWMA for PNF Timing Info

This scenario validates the application of an Exponentially Weighted Moving Average (EWMA) to smooth the PNF Timing Info. Without EWMA processing, the raw timing information exhibits high variance due to network jitter, as shown in Fig. 3(a). Applying EWMA effectively filters out this high-frequency jitter (Fig. 3(b)), providing a stable baseline for the dynamic timing adjustment algorithm. Furthermore, as illustrated in Fig. 3(c) and Fig. 3(d), the smoothed data enables the system to reliably detect downward trends in the arrival offset (Δt_{arrive}). Therefore, when Δt_{arrive} begins to exhibit a downward trend, the algorithm can stably judge this condition and actively advance more ahead time to maintain synchronization.

D. Scenario II: Performance Benchmark

We benchmark the peak throughput and MAC HARQ RTT against a static baseline. Under stable fronthaul conditions, as shown in Fig. 4, the proposed algorithm prioritizes guaranteeing the optimal throughput under different offered loads, consistently maintaining the best throughput performance. Simultaneously, while ensuring this optimal throughput, the algorithm also maximizes the selection of the best slot ahead to optimize the MAC layer HARQ Round-Trip Time (RTT), effectively avoiding unnecessary overhead.

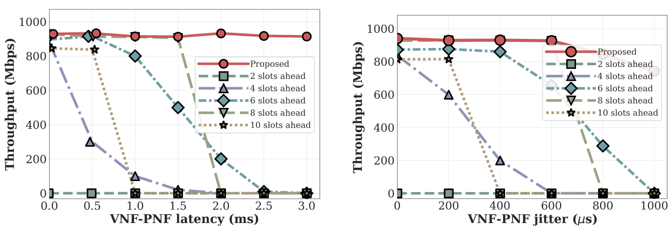


(a) Throughput Optimization (b) MAC Latency & Reliability

Fig. 4. Performance evaluation: MAC RTT efficiency and system throughput.

E. Scenario III: Stability under Network Congestion

To evaluate stability, we inject delay and jitter between the VNF and PNF using Linux TC. As shown in Fig. 5a, static slots ahead configurations suffer from synchronization failures and throughput drops when delay exceeds the timing window, whereas our proposed adaptive mechanism dynamically increases the slots ahead to maintain stable throughput. Additionally, we evaluate the UE DL RX throughput under a saturated 1 Gbps UDP offered load with unpredictable link jitter. As shown in Fig. 5b, while fixed slot ahead configurations experience severe throughput degradation due to late packets, the proposed adaptive method dynamically tracks jitter and secures stable, high throughput at the UE.



(a) Stability under Delay Congestion

(b) Throughput under Unpredictable Jitter

Fig. 5. System stability and throughput performance under varying network conditions.

V. Conclusion

This paper presented a dynamic timing adjustment framework for disaggregated RAN architectures, specifically focusing on the nFAPI-based Split Option 6 over non-ideal packet-switched networks. Recognizing the limitations of static slots ahead configurations under unpredictable network jitter and server processing delays, we proposed an Adaptive Slots Ahead mechanism based on Exponentially Weighted Moving Average (EWMA) to dynamically track and reduce network latency variations. Experimental validations conducted on the OpenAirInterface (OAI) platform with a commercial Pegatron O-RU demonstrated that the proposed method effectively eliminates synchronization failures and maintains stable system throughput, even under severe network congestion. Unlike existing solutions that rely on expensive hardware-level synchronization or specialized acceleration, our software-based approach offers a cost-effective and flexible alternative for deploying robust O-RAN systems on general-purpose Commercial Off-the-Shelf (COTS) servers.

References

[1] *TR 38.801 Study on new radio access technology: Radio access architecture and interfaces*, 3rd Generation Partnership Project (3GPP), 2017, version 14.0.0. [Online]. Available: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3066

[2] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. D. Renzo, T. Melodia, X. Costa-Pérez, M. Debbah, A. Dutta, and H. D. Schotten, "A comprehensive tutorial and survey of o-ran: Exploring slicing-aware architecture, deployment options, use cases, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1637–1678, 2026.

[3] O-RAN Alliance, "O-ran fronthaul working group control, user and synchronization plane specification," O-RAN Alliance, Tech. Rep. O-RAN.WG4.CUS.0-v11.00, 2023.

[4] Small Cell Forum (SCF), "5g network fapi (nfapi) specification," Small Cell Forum (SCF), Tech. Rep. SCF225, 2020. [Online]. Available: https://www.smallcellforum.org/docs/5g-nfapi-specifications/

[5] X. Foukas *et al.*, "Computational resource consumption of 5g virtualized ran," in *2021 IEEE International Conference on Communications (ICC)*. IEEE, 2021.

[6] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, "X5g: An open, programmable, multi-vendor, end-to-end, private 5g o-ran testbed with nvidia arc and openairinterface," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11 305–11 322, 2025.

[7] J. L. Herrera, S. Montebugnoli, D. Scotece, L. Foschini, and P. Bellavista, "A tutorial on O-RAN deployment solutions for 5G: From simulation to emulated and real testbeds," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1709–1748, 2026.

[8] S.-Q. Lee and M.-S. Lee, "A method of adaptive low-latency downlink transmission on fapi based 5g nr gnb," in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2022.

[9] Y. Jia, Y. Zhong, M. Wang, J. Gao, P. Zhang, X. Liu, and X. Jin, "Aquifer: Transparent microsecond-scale scheduling for vRAN workloads," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3171–3184, 2024.

[10] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, A. Garcia-Saavedra, and X. Costa-Pérez, "Oranus: Latency-tailored orchestration via stochastic network calculus in 6g o-ran," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 61–70.

[11] J. Lv, Y. Gao, Z. Ding, Y. Lin, X. You, G. Yang, and W. Dong, "Providing ue-level qos support by joint scheduling and orchestration for 5g vran," in *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*, 2024, pp. 51–60.

[12] M. Elfiky, Z. Becvar, and P. Mach, "Allocation of resources for HARQ retransmission in mobile networks based on C-RAN," *IEEE Systems Journal*, vol. 18, no. 1, pp. 450–461, 2024.

[13] E. S. Hidalgo, J. A. Ayala-Romero, J. X. Salvat Lozano, A. Garcia-Saavedra, and X. C. Perez, "AegisRAN: A fair and energy-efficient computing resource allocation framework for vRANs," *IEEE Transactions on Mobile Computing*, vol. 24, no. 10, pp. 9441–9457, 2025.

[14] L. Diez, A. M. Alba, W. Kellerer, and R. Agüero, "Flexible functional split and fronthaul delay: A queuing-based model," *IEEE Access*, vol. 9, pp. 151 049–151 066, 2021.

[15] F. Khan, O. Gil, L. Diez, E. S. Santiago, L. M. Contreras, and R. Agüero, "Evaluating fronthaul network performance under the O-RAN paradigm: A novel methodology based on queuing theory," *IEEE Transactions on Network and Service Management*, vol. 23, pp. 1723–1741, 2026.

[16] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 32–40, 2018.

[17] X. Wang *et al.*, "An open-source implementation of the 5g nr functional split option 6," in *2022 IEEE International Conference on Communications (ICC)*, 2022, pp. 1–6.

[18] O-RAN Alliance, "O-ran architecture description," O-RAN Alliance Working Group 1, Tech. Rep. O-RAN.WG1.O-RAN-Architecture-Description, 2024.